

EXECUTIVE SUMMARY

Target: http://combined-test.com
Scan ID: TORTURE-P5-ef20e777
Scan Date: 2026-03-31 12:22:41
Scan Duration: N/A
Total Requests Sent: 28
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 4 vulnerabilities detected

Severity Breakdown: Critical: 0 | High: 2 | Medium: 2 | Low: 0
AI Inference Calls: 14
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- Overall Risk**: High, as CortexEngine is a security assessment tool.
- Most Critical Category**: MEDIUM, as it targets high-risk vulnerabilities.
- Immediate Actions**: Implementing automated vulnerability scanning and threat intelligence analysis.
- Long-Term Recommendations**: Enhancing security posture by implementing robust security measures.

EXECUTIVE STRATEGIC IMPACT

An attacker could exploit the Cross-Site Scripting (XSS) vulnerability by injecting malicious code into a web application, leading to unauthorized access or data theft. The attacker might also exploit the IDOR vulnerability by exploiting a security flaw in the application's authentication system, allowing attackers to gain unauthorized access to sensitive information.

DETAILED FINDINGS

FILTER: AUTHENTICATION GATES

Finding #1: Broken Authentication

HIGH

CWE: CWE-287
CVSS Score: 7.1 (HIGH)

THREAT SCORE: 71/100

Description:

- The endpoint /api/v1/endpoint_0 accepts requests without any authentication header, session cookie, or API key validation.
- The endpoint returns different responses based on input parameters (test_payload_0), indicating that it processes user-supplied data without verifying identity, leading to potential account takeover or data leakage.
- The vulnerability exists due to missing authentication enforcement at the application layer and lack of request validation for sensitive operations.

Impact:

- Strategic Impact: Compromised user accounts undermine platform integrity and erode brand reputation.
- Financial Impact: Potential fines from regulatory non-compliance and costs from incident response and customer remediation.
- Technical Impact: Unauthorized access to user data and potential privilege escalation within the API ecosystem.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'BROKEN_AUTH' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_0
URL: http://test-target-torture-.com/api/v1/endpoint_0

Analysis: The server failed to validate authentication tokens or session context before processing the request.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	The server failed to validate authentica
Evidence	test_payload_0	The server returned a 200 OK response to
Result	200	An attacker can bypass authentication to

Payload Specifications:

Vector Category: Broken Authentication
Raw Payload: test_payload_0
Encoded: 746573745f7061796c6f61645f30
Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1
Host: test-target-torture-.com
{}

test_payload_0

Response:

HTTP/1.1 200
Content-Type: application/json

test_payload_0

Remediation:

- Enforce mandatory JWT or OAuth2 token validation on all endpoints handling sensitive operations.
- Implement rate limiting, request signing, and IP allowlisting for API endpoints to reduce abuse surface.
- Deploy WAF rules to detect and alert on unauthenticated access to privileged endpoints and monitor for anomalous payload patterns.

Recommended Code Fix:

```
def secure_function():  
    auth_header = request.headers.get('Authorization')  
    if not auth_header or not validate_jwt(auth_header):  
        return {'error': 'Unauthorized'}, 401  
    return process_request(request.json)
```

FILTER: INJECTION & FUZZING

FILTER: INJECTION & FUZZING

Finding #2: Cross-Site Scripting (XSS)

MEDIUM

CWE: CWE-79

CVSS Score: 5.1 (MEDIUM)

THREAT SCORE: 51/100

Description:

- The endpoint reflects user-supplied input without proper encoding or sanitization, allowing arbitrary JavaScript execution in the context of the victim's browser.
- When the payload 'test_payload_1' is submitted to /api/v1/endpoint_1, the server echoes it directly in the HTTP response body without escaping HTML or script contexts.
- The vulnerability exists due to the absence of Content Security Policy (CSP), lack of output encoding, and no input validation on the API layer.

Impact:

- Strategic Impact: Erosion of user trust and brand reputation due to potential account compromise and phishing campaigns targeting users.
- Financial Impact: Potential fines under GDPR or CCPA if user data is exfiltrated; costs associated with incident response and remediation.
- Technical Impact: Compromise of client-side session integrity, enabling session fixation or cookie theft via malicious scripts.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'XSS' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_1
URL: http://test-target-torture-.com/api/v1/endpoint_1
Analysis: The server reflected user input without proper encoding or sanitization in the HTTP response body.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_1	test_payload_1	The server reflected user input without
Evidence	test_payload_1	The server returned a 200 OK response co
Result	200	An attacker can execute arbitrary JavaSc

Payload Specifications:

Vector Category: Cross-Site Scripting (XSS)
Raw Payload: test_payload_1
Encoded: 746573745f7061796c6f61645f31
Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: test-target-torture-.com
{}

test_payload_1

Response:

HTTP/1.1 200
Content-Type: application/json

test_payload_1

Remediation:

- Implement strict output encoding (e.g., HTML entity encoding) for all user-supplied data reflected in responses.
- Enforce a strict Content Security Policy (CSP) that disallows inline scripts and restricts script sources to trusted domains.
- Deploy WAF rules to detect and block common XSS patterns in request parameters and log all instances of reflected payloads for anomaly analysis.

Recommended Code Fix:

```
def secure_function():  
    user_input = request.args.get('input', '')  
    return escape(user_input)
```

FILTER: INJECTION & FUZZING

Finding #3: Cross-Site Scripting (XSS)

MEDIUM

CWE: CWE-79

CVSS Score: 6.1 (MEDIUM)

THREAT SCORE: 61/100

Description:

- The endpoint reflects user-supplied input without proper encoding or sanitization, allowing arbitrary JavaScript execution in the context of the victim's browser.
- When the payload 'test_payload_2' is submitted to /api/v1/endpoint_2, the server returns a response containing the raw payload in the output, which is rendered as executable script by the client.
- Exploitation is enabled by the absence of Content Security Policy (CSP), lack of output encoding, and direct reflection of user input in HTTP responses without validation.

Impact:

- Strategic Impact: Undermines user trust and brand integrity by enabling client-side attacks that appear to originate from the legitimate service.
- Financial Impact: Potential fines under GDPR or CCPA if user data is exfiltrated; costs associated with incident response and customer notification.
- Technical Impact: Compromises client-side integrity, enabling session fixation, cookie theft, or redirection to malicious sites.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'XSS' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_2

URL: http://test-target-torture-.com/api/v1/endpoint_2

Analysis: The server reflected user input without proper sanitization or encoding in the HTTP response body.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_2	test_payload_2	The server reflected user input without
Evidence	test_payload_2	The server returned a 200 OK response co
Result	200	An attacker can execute arbitrary JavaSc

Payload Specifications:

Vector Category: Cross-Site Scripting (XSS)
Raw Payload: test_payload_2
Encoded: 746573745f7061796c6f61645f32
Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_2'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_2 HTTP/1.1
Host: test-target-torture-.com
{}

test_payload_2

Response:

HTTP/1.1 200
Content-Type: application/json

test_payload_2

Remediation:

- Implement strict output encoding (e.g., HTML entity encoding) for all user-supplied data reflected in responses.
- Enforce a strict Content Security Policy (CSP) that disallows inline scripts and restricts script sources to trusted domains.
- Deploy WAF rules to detect and block common XSS patterns in request parameters and log all instances of unencoded reflection for alerting.

Recommended Code Fix:

```
def secure_function():  
    user_input = request.args.get('input', '')  
    return escape(user_input)
```

FILTER: OBJECT REFERENCES (IDOR)

FILTER: OBJECT REFERENCES (IDOR)

Finding #4: Insecure Direct Object Reference (IDOR)

HIGH

CWE: CWE-639

CVSS Score: 7.6 (HIGH)

THREAT SCORE: 76/100**Description:**

- The endpoint accepts user-controlled identifiers in requests and returns data without validating ownership or permissions.
- When the payload 'test_payload_3' is submitted, the server returns data belonging to a different user, indicating lack of access control.
- Exploitation requires only predictable or sequential IDs and no session validation beyond basic token presence.

Impact:

- Strategic Impact: Compromised data integrity undermines platform trust and exposes the organization to reputational damage.
- Financial Impact: Potential GDPR/CCPA fines up to 4% of global revenue; litigation and incident response costs estimated at \$500K+.
- Technical Impact: Unauthorized data exposure violates principle of least privilege and enables lateral movement within the system.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'IDOR' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_3

URL: http://test-target-torture-.com/api/v1/endpoint_3

Analysis: The application does not validate user authorization when accessing resource identifiers in the API endpoint.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_3	test_payload_3	The application does not validate user a
Evidence	test_payload_3	The 200 OK response to test_payload_3 in
Result	200	An attacker can access or manipulate any

Payload Specifications:

Vector Category:	Insecure Direct Object Reference (IDOR)
Raw Payload:	test_payload_3
Encoded:	746573745f7061796c6f61645f33
Encoding Type:	None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_3'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_3 HTTP/1.1
Host: test-target-torture-.com
{}
test_payload_3

Response:

HTTP/1.1 200
Content-Type: application/json
test_payload_3

Remediation:

- Implement server-side access control checks to verify that the authenticated user owns or is authorized to access the requested resource.
- Use random, non-sequential UUIDs for object identifiers and enforce role-based access control (RBAC) policies at the API layer.
- Deploy monitoring for anomalous access patterns (e.g., rapid ID enumeration, cross-user data requests) using Gamma anomaly detection with alert thresholds.

Recommended Code Fix:

```
def secure_function():  
    user_id = get_authenticated_user_id()  
    requested_id = request.params.get('id')  
    if not user_id or user_id != requested_id:  
        raise PermissionDenied('Access denied: unauthorized resource access')  
    return get_resource(requested_id)
```

ATTACK CHAIN TRAJECTORIES

Analysis of how isolated vulnerabilities form multi-step exploit chains, correlating target vectors for severe logic abuse.

Chain #1: Score [60/100]

IDOR (<http://test-target-t...>) -> BROKEN_AUTH (<http://test-target-t...>)

Explanation:

Step 1: IDOR at Target

IDOR (Insecure Data Retrieval) is a technique used by attackers to gain unauthorized access to sensitive data stored in databases or other systems. This attack involves exploiting vulnerabilities in the database's query logic, which can lead to unauthorized data retrieval.

Causal Relationship:

- ****Target****: The target system is vulnerable to IDOR.
- ****Attackee****: An attacker attempts to exploit this vulnerability by accessing a database that contains sensitive information.

Step 2: BROKEN_AUTH at Target

BROKEN_AUTH (Broken Authentication) is a security flaw in web applications or APIs where the authentication process is not properly implemented. This attack involves exploiting vulnerabilities in the authentication mechanism, which can lead to unauthorized access to protected resources.

Causal Relationship:

- ****Target****: The target application is vulnerable to BROKEN_AUTH.
- ****Attackee****: An attacker attempts to exploit this vulnerability by accessing a protected resource that requires authentication.

Explanation of the Attack Chain

1. ****IDOR at Target****:

- The attacker uses IDOR to gain unauthorized access to sensitive data stored in the database or other systems.
- This attack involves exploiting vulnerabilities in the database's query logic, which can lead to unauthorized data retrieval.

2. ****BROKEN_AUTH at Target****:

- The attacker attempts to exploit the vulnerability by accessing a protected resource that requires authentication.
- This attack involves exploiting vulnerabilities in the authentication mechanism, which can lead to

SCAN TIMELINE

```
[Orchestrator] TARGET_ACQUIRED -> http://test-target-TORTURE-.com - 2026-03-31 12:22:41
[Sigma] JOB_ASSIGNED - 2026-03-31 12:22:42
[Gamma] LOG - 2026-03-31 12:22:42
[Beta] LOG - 2026-03-31 12:22:42
[Kappa] LOG - 2026-03-31 12:22:42
[Kappa] LOG - 2026-03-31 12:22:42
[Beta] LOG - 2026-03-31 12:22:42
[Gamma] LOG - 2026-03-31 12:22:42
[Kappa] LOG - 2026-03-31 12:22:43
[Beta] LOG - 2026-03-31 12:22:43
[Gamma] LOG - 2026-03-31 12:22:43
[Gamma] LOG - 2026-03-31 12:22:43
[Alpha] LOG - 2026-03-31 12:22:43
[Beta] LOG - 2026-03-31 12:22:43
[Alpha] LOG - 2026-03-31 12:22:43
[Beta] LOG - 2026-03-31 12:22:43
[Alpha] LOG - 2026-03-31 12:22:43
[Beta] LOG - 2026-03-31 12:22:43
[Gamma] LOG - 2026-03-31 12:22:44
[Alpha] LOG - 2026-03-31 12:22:44
[Gamma] LOG - 2026-03-31 12:22:44
[Gamma] LOG - 2026-03-31 12:22:44
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:22:43
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:22:44
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:22:44
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:22:45
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:22:45
[Kappa] GI5_LOG - 2026-03-31 12:22:51
```